REMORA: Real-time Expressive Motion Routing for Audio

Timothée Dravet
Polytech Marseille, Aix-Marseille Université
France
timothee.dravet@etu.univ-amu.fr

Daniel Overholt
CREATE, Aalborg University
Denmark
dano@create.aau.dk

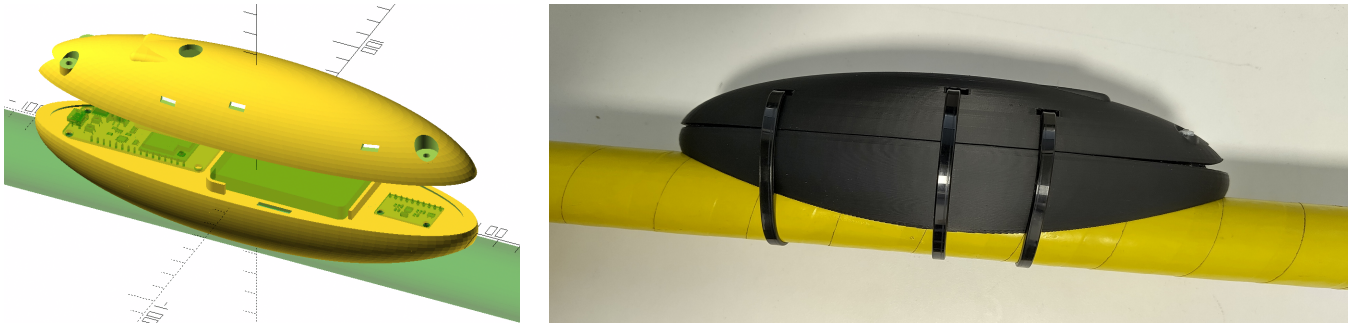


Figure 1: The REMORA sensor unit: parametric OpenSCAD enclosure (left) and the printed enclosure zip-tied near one end of the bô (right).

Abstract

REMORA is a martial-arts staff (bô) played as a musical instrument. An inertial sensor strapped near one end streams over Wi-Fi to an audio plugin, which turns spins, turns of the body, and thrusts into sound. We wanted features that belong to the staff rather than to IMUs in general, so the plugin tracks the plane and direction of a spin, which of two floor lines the staff is working along, how many laps a spin has lasted, and, experimentally, thrusts along the staff. A three-pose calibration makes all of this independent of how the enclosure is rotated on the staff. The mapping from features to sound is a small dataflow patch that can be edited inside the plugin while audio runs. We use it as an authoring tool in rehearsal, not as something the performer rewires mid-form. We also report what went wrong. A pilot with ten mostly novice players liked the instrument but found thrusts poorly detected. Tracing that led us to three defects: the analyzer ran per audio block, gravity was subtracted in the wrong frame, and the orientation filter lost tilt during fast spins. We describe the fixes and check them in simulation. Measurements on the real staff and sessions with trained players are still to come.

CCS Concepts

• **Applied computing** → **Sound and music computing**;
• **Software and its engineering** → *Visual languages.*

Keywords

digital musical instrument, martial arts, bô staff, inertial sensing, gesture mapping, visual patching, OSC

This work is licensed under a Creative Commons Attribution 4.0 International License.

1 Introduction

1.1 Motivation

The first author trains with the bô and also makes music, and REMORA started as an attempt to make one practice audible through the other. The bô is a staff of about 1.5 m. It is held in two hands and swung, spun, and thrust through techniques that are taught as forms (*kata*). Most inertial-sensor Digital Musical Instruments (DMIs) shown at New Interfaces for Musical Expression (NIME) are small: something held in one hand or worn on the body. A staff is different. It is long, it has real rotational inertia, it reaches far from the body, and a trained player already controls a large, codified set of movements with it.

We did not know at the start which of those movements should make which sounds, and we still do not fully know. Our first answer was to write each gesture-to-sound mapping as its own C++ class. That made every experiment a rebuild. After eleven classes we noticed that they kept reusing the same arithmetic, the same smoothing, and the same few stateful tricks, so we moved the mapping into a dataflow patch that is edited inside the audio plugin itself. Visual patching is not new (§2). For us it is the tool we use to try out and revise the staff's vocabulary.

1.2 Contributions

(1) Motion features designed for a long, swung object: spin plane and direction, a two-state facing that needs no magnetometer, and a continuous lap count, all computed once per sensor packet from the gyroscope in a calibrated world frame (§4.3). An axial thrust detector is included but has not yet been validated on the real staff.
(2) A three-pose calibration that removes the effect of how the enclosure is rotated on the staff, including the zero of roll (§4.2).
(3) A mapping layer inside the plugin, built as a typed directed acyclic graph (DAG) over a catalog of 69 node types, in

which all eleven earlier hand-coded mappings and four new ones are expressed as presets (§4.4, §4.5).

We also describe three defects that the pilot study exposed and how we fixed them (§6.2), because two of them would affect any staff- or club-like instrument that reuses a stock orientation filter. Table 3 lists the evidence for each contribution and what is still untested.

1.3 Scope

We designed REMORA for a soloist with some staff or martial-arts background who plays a set built from presets. The presets are graded. *Simple* needs only tilt and roll. The facing- and spin-driven *Martial Momentum* and *Azimut Kinetic* only make sense if the player can already spin and turn with a staff. The patch editor is for whoever prepares the set: the performer in rehearsal, or a second person at the computer during a performance. Ensemble use is out of scope.

2 Related Work

2.1 Rod, Baton, and Swung Controllers

The *T-Stick* [11] senses touch and pressure along its shaft together with accelerometer-derived shake and tap gestures. Conducting interfaces include Mathews' *Radio Baton* [14], Marrin and Paradiso's *Digital Baton* [12], and the *Conductor's Jacket* [13]. Nam's *mPoi* [15] made a swung object into a DMI and builds its sound from the path and momentum of the swing. *x-OSC* [8] popularised the Wi-Fi/OSC inertial streaming pattern that REMORA's sensor unit follows.

2.2 Martial Arts and Object Manipulation

Takahata et al. [22] gave karate beginners wearable sound feedback while they practised the basic punch, *tsuki*, which is the same gesture our thrust detector targets. Their goal was training rather than performance, and they found that the sound made practice more enjoyable. Juggling is a closer cousin of staff work in one respect: it is a skilled object practice with its own failure modes. Özcan and Çamcı [17] use it to make failure itself musical material. In our search of the NIME proceedings we did not find a performance instrument built on a martial-arts weapon such as a staff, sword, or nunchaku. That search was not exhaustive, and we would welcome pointers.

2.3 Mapping Tools and Visual Patching

Visual dataflow patching descends from Puckette's Patcher, now *Max* [18, 20], and continues in *Pure Data* [19] and *VCV Rack* [1]. For DMIs, libmapper [10] and its graphical front end Webmapper [26] connect controller signals to synthesis parameters across a network, and *ossia score* [3] combines such mappings with a timeline. Any of these could host REMORA's mappings. We put the graph inside the plugin for practical reasons: mapping and synthesis share one process and one preset file, and the patch shows each signal's live value next to the synthesiser input it drives (§4.6). The T-Stick with libmapper is the closest prior system. Where REMORA differs is the object. A full-length swung staff has a spin plane, a spin direction, and a line it works along, and a handheld rod's touch and shake gestures have nothing that corresponds to them.

2.4 Mapping Strategies

Hunt and Wanderley [7] classify mappings as one-to-one, one-to-many, or many-to-one, and argue that richer mappings make an interface feel more like an instrument. Van Nort et al. [24] extend mapping to continuous navigation between parameter spaces, whereas REMORA's graph stays with explicit parameter-level functions. *Wekinator* [5] and Françoise et al. [6] learn mappings from example gestures, so the performer does not have to wire them. Expressive-movement systems often describe motion with Laban's Effort factors [2, 21], and Fdili Alaoui et al. [4] show that capturing those well takes expert annotation and multimodal data. We deliberately stay with plain kinematic signals tied to the staff.

3 Design

Five requirements shaped the design:

- **A real bô.** The instrument keeps the reach, weight, and two-handed grip of a full-length staff.
- **Wireless during performance.** A cable is acceptable only for charging.
- **Live-editable mapping.** Loading or rewiring a mapping must not interrupt audio.
- **Mount-independent calibration.** The sound should not depend on how the enclosure is rotated around the staff.
- **Low latency.** The sound should follow the motion closely enough to feel driven by it (§6.4).

The firmware estimates orientation and sends Open Sound Control (OSC) packets over Wi-Fi. The plugin does everything after that: reception, calibration, motion analysis, graph evaluation, and synthesis.

4 Implementation

4.1 Hardware and Firmware

The sensor unit is a SparkFun ESP32-S2 Thing Plus reading an MPU-9250 inertial measurement unit (IMU) over I2C at 400 kHz, powered by a 2000 mAh LiPo battery (Figure 2). It sits in a parametric OpenSCAD enclosure sized to the staff (Figure 1) and is zip-tied near one end, outside the grip, so it can be moved or removed without damaging the staff. The enclosure holds the board flat against the staff, so one board axis always runs along it. Which axis that is, and how the board is rolled around the staff, is left to calibration. The position along the staff does matter (§4.3) and should stay the same between sessions.

The IMU is read at 200 Hz with ranges of ±16 g and ±2000°/s; fast spins come close to the gyroscope limit. Orientation is estimated on the microcontroller by a 6-axis Mahony filter [9]. The magnetometer is not fused, because next to the staff's own electronics and stage equipment it is unreliable. Our first version used the fixed-gain filter of the `hideakitai/MPU9250` library ($K_p = 30$), which trusts the accelerometer as a gravity reference at all times. On a staff that is a poor assumption. With the unit about $r = 0.7$ m from the hands, a 2 rev/s spin adds a centripetal acceleration of $\omega^2 r \approx 11$ g. The current filter scales K_p by two weights: one that falls from 1 to 0 as $\big||a| - 1\,\mathrm{g}\big|$ grows from 0.1 to 0.3 g, and one that falls from 1 to 0 as the rotation rate grows from 90 to 180°/s. During fast motion the filter therefore integrates the gyroscope alone and corrects tilt

once the staff slows down. The gyroscope bias is averaged at boot and then tracked whenever the staff has been still for half a second. Heading has no absolute reference, so this tracking is the only defence against yaw drift.

Each packet carries the quaternion, the raw acceleration, angular rate, and magnetic field, a sequence number, and the sensor's clock in microseconds, and is sent over UDP every 10 ms. The host computer runs a Wi-Fi hotspot, the staff joins it, and packets go to the network's gateway, which is the host. The plugin checks every argument's type, drops duplicate or out-of-order packets, and counts lost ones from the sequence number. It shows inter-arrival time and network jitter (the spread of arrival time minus sensor time) live, and it can log every packet to a file.



Figure 2: Inside the enclosure: ESP32-S2, MPU-9250 IMU, and LiPo battery.

4.2 Calibration

Orientation is a unit quaternion, and rotating v by q gives $v' = q\,v\,q^{-1}$. The performer is prompted through three poses: A, staff horizontal pointing forward; B, staff vertical pointing up; and C, staff horizontal pointing right. These give quaternions q_A, q_B, q_C.

Stage 1: which board axis runs along the staff. Each candidate $L \in \{\pm X, \pm Y, \pm Z\}$ is scored by

$$v_P = \widehat{\text{rotate}(L, q_P)}\,, \qquad e(L) = |v_A{\cdot}v_B| + |v_B{\cdot}v_C| + |v_C{\cdot}v_A|\,.$$

Only the staff axis is carried through three mutually orthogonal directions by the three poses, so it scores lowest. Because the search covers only the six board axes, it relies on the enclosure holding one of them along the staff. L and $-L$ score the same. The tip is the one that points forward, up, and right in A, B, C, and physically forward × up = right, so the tip's triple product $(v_A \times v_B)\cdot v_C$ should be +1. In our frames it is −1, and the reason is a convention we inherited. The library feeds its filter $(-a_x, a_y, a_z)$ and $(g_x, -g_y, -g_z)$, which mirrors the chip's x axis. The quaternion is therefore a proper rotation between two mirror images of physical space. Every vector the plugin uses lives in that mirrored frame, so everything stays consistent with everything else, but cross products change sign. Our filter keeps the same input convention so that existing calibrations and presets stay valid. Hence

$$L^* = \underset{L\,:\,(v_A\times v_B)\cdot v_C < -0.5}{\arg\min} \; e(L)\,,$$

which gives the alignment quaternion q_{align}. If no axis qualifies, or if $e(L^*) > 1.5$ (roughly, some pair of poses more than 30° from a right angle), calibration is rejected and the performer is asked to try again.

Stage 2: heading and tilt. Let b_0 and b_1 be the measured staff direction in poses A and B, and $n = \widehat{b_0 \times b_1}$. The triad $U = [\, b_0,\ n,\ (b_0 \times n)\,]$ is built from them, and W likewise from the instrument-frame forward $(1, 0, 0)$ and up $(0, 0, 1)$. The rotation $R = WU^\top$ takes the measured directions onto the instrument's, and it is converted to a quaternion q_{corr}.

Stage 3: zero of roll. Stages 1 and 2 fix where the staff points but leave its roll at whatever angle the enclosure happens to sit around the staff. That matters for presets driven by roll (*Simple*, *Spin Voices*). A last rotation q_{roll} about the staff axis makes pose A read as zero roll. Every reading is then mapped to instrument space by

$$q' = q_{\text{corr}}\; q_{\text{raw}}\; q_{\text{align}}\; q_{\text{roll}}\,,$$

so roll is measured from how the performer held the staff in pose A, not from how the box was strapped on. The angular rate and acceleration are rotated into the same frame. A "re-zero heading" button reuses q_{corr} to make the staff's current direction the new forward without repeating the poses. The position of the unit along the staff is not corrected.

4.3 Motion Features

One analyzer computes the features in Table 1, and every mapping graph reads from it. It runs once per packet, in order, with Δt taken from the sensor's clock (§6.2). Smoothing rates and decays were tuned by ear with one update per 256-sample block at 48 kHz. They are now written per *reference tick* of $T = 5.33$ ms and converted to the actual step, so that $\alpha' = 1-(1-\alpha)^{\Delta t/T}$ for a one-pole coefficient and $d' = d^{\,\Delta t/T}$ for a decay.

Table 1: Motion features computed by the analyzer.

Feature	Purpose
Tip vector, rotation axis	where the staff points; what it spins about
Dynamic acceleration	motion with gravity removed
Energy, onset, axis stability, jerk	how hard and how steadily the staff moves
Spin plane and direction	vertical/horizontal spin, CW/CCW
Facing	along or across the forward line
Continuous spin count	full turns in the current spin
Axial thrust (experimental)	stab or strike along the staff

Tip, axis, and gravity. The tip direction is $p = \text{rotate}((1, 0, 0), q')$. The rotation axis comes straight from the gyroscope, rotated into the world frame as ω, with the roll about the staff removed: $\omega_\perp = \omega - p\,(p \cdot \omega)$. Rolling the staff in the hands is not a spin, and the lap count accumulates $\|\omega_\perp\|\,\Delta t$, not $\|\omega\|\,\Delta t$. Because the specific force is also in the world frame, the dynamic acceleration is simply $a_{\text{dyn}} = a_{\text{world}} - (0, 0, 1)$ g.

Energy features. Four smoothed signals describe how hard the staff is moving. *Energy* is the enveloped magnitude of a leaky integral of a_{dyn}. *Onset* is the positive rate of change of rotation speed. *Axis stability* is the smoothed absolute dot product between consecutive rotation axes; it is high while the staff keeps turning about the same axis. *Jerk* is the smoothed jerk of $|a_{\text{dyn}}|$, given both as an "abrupt" value and as its complement. During spins, centripetal acceleration dominates a_{dyn}, so Energy and Jerk mostly follow spin speed. That is how the presets use them, and it is also why their scale changes if the unit is moved along the staff.

Spin plane and direction. Let (a_x, a_y, a_z) be $\omega_\perp$ after one-pole smoothing. The spin is *vertical* (the staff turns in a vertical plane, like a baton twirled beside the body) when

$$\sqrt{a_x^2 + a_y^2} > |a_z|\,,$$

and *horizontal* (like a propeller overhead) otherwise. For a horizontal spin, the direction is the sign of a_z. For a vertical spin it is the sign of the larger of a_x, a_y. In the *Martial* presets it is instead the sign of the axis' dot product with a reference axis latched at the first vertical spin after the preset loads.

Facing. A compass heading would need the magnetometer. Facing is instead a two-state feature: is the staff working *along* the calibration's forward line or *across* it? It does not tell front from back or left from right, and we no longer call the states "north" and "east". During a vertical spin, the spin plane decides. When the staff is not turning about the vertical (rotation about z below 60°/s) and is within 60° of horizontal, the direction it points decides. In every other case the state is held. That last rule is important. In our first version, an overhead spin fell back on the tip direction, which sweeps through every heading, so facing toggled every quarter turn. Let (d_x, d_y) be the horizontal normal of the line the staff works along: the smoothed rotation axis in a vertical spin, or the tip direction turned by 90° otherwise. With hysteresis factor $k = 1.15$, the state switches

$$\text{across} \to \text{along if } |d_y| > k\,|d_x|\,, \qquad \text{along} \to \text{across if } |d_x| > k\,|d_y|\,,$$

that is, at $\arctan(1.15) \approx 49°$ and $\arctan(1/1.15) \approx 41°$. Facing is only as good as the heading. Gyroscope bias is tracked (§4.1) and heading can be re-zeroed, but we have not yet measured how far it drifts over a set.

Axial thrust (experimental). A thrust is flagged when the jerk of $a_{\text{dyn}} \cdot p$ crosses zero while $|a_{\text{dyn}} \cdot p|$ exceeds 1.5 g and the staff is turning slower than 90°/s, followed by a 0.2 s cooldown. It failed in the pilot. The causes we found and fixed are in §6.2. It passes our simulated tests (§6.3), but we will not count it as a contribution until it has been tested on labelled thrusts with the real staff.

4.4 Mapping Architecture

The node catalog has four categories: Source (sensor values and the features above), Math, Sink (one node per synthesis engine exposing all its parameters as typed inputs, plus output gain), and Display (live monitors). A graph is a set of nodes plus typed wires, saved as an XML preset. The graph is the only path from sensor to synthesiser.

When a wire is added, the graph recomputes a topological order with Kahn's algorithm and refuses the wire if it would close a cycle. For every packet, evaluation walks the cached order once, and each node computes $y_i = f_{\text{type}(n_i)}(x_{i,1}, \ldots, x_{i,m}; \theta_i)$ from its inputs x and parameters θ. The patch is thus one composed function of the sensor stream, with no sorting on the audio thread. If the editor holds the graph when a packet arrives, evaluation does not wait; the synthesiser keeps the previous values.

Math nodes cover arithmetic, shaping (linear and logarithmic range mapping, clamp, threshold, crossfade, quantise, sine, atan2, semitones to Hz), lookup tables (1-D, and an 8-entry table indexed by three booleans), and stateful dynamics (one-pole smoother, leaky integrator, retrigger envelope, hysteresis, derivative, latch, random sample-and-hold). The stateful nodes use the same reference tick as the analyzer, so a patch behaves the same at any buffer size.

4.5 Presets

The eleven C++ mappings from development were rebuilt as graph presets: *Simple*, *Lead+Drone*, *Spin Filter*, *Bowed Chord*, four *Azimut* variants, *Martial Effort*, *Martial Momentum*, *Spin Voices*, and *Speed Gate*. Four more were made directly as graphs: *Wind Noise*, *Azimut Kinetic*, *Spin Voices (Scale)*, and *Vocal Tract*. Nine are offered in the plugin's selector (Table 2); the others are close variants kept as reference patches.

Table 2: The nine presets in the plugin's selector.

Preset	Character
Simple	Pitch from tilt, volume from roll; one sine voice.
Wind Noise	No pitch; motion energy swells filtered noise into gusts.
Bowed Chord	Root from tilt; yaw and roll pick the chord; spin speed acts like bow pressure.
Lead+Drone	Major-scale lead over a 4-voice drone.
Martial Momentum	Root from spin plane and direction; glide speed from spin momentum.
Azimut Kinetic	Root from spin and facing; rotation speed drives the whole timbre.
Speed Gate	Slow: pentatonic melody. Fast: crossfades to the Azimut mapping.
Spin Voices (Scale)	Roll selects 1 of 4 scale-quantised voices.
Vocal Tract	Physically modelled voice; jabs add consonant-like bursts.

The catalog has 69 node types (33 source, 29 math, 4 sink, 3 display). Presets range from 7 nodes (*Simple*) to 178 (*Martial Effort*), with a median of 86. Together they use 50 of the 69 types, and most of the unused ones are raw per-axis sensor values. No preset needed a node type of its own. That shows the catalog is enough for these mappings, but the catalog and the mappings grew together, so it does not show that someone else's mappings would fit. Per-preset counts are generated from the code by a tool in the repository. We checked the rebuilt presets against the original classes by ear only. The fixes in §6.2 have since changed the features underneath both, so an exact comparison with the old classes is no longer meaningful. Instead, any two versions can now be compared by replaying the same packet log through them.

Azimut root table. The Azimut presets and the fast branch of *Speed Gate* choose the root note from three booleans (vertical spin, counter-clockwise, working along the forward line) through the 8-entry lookup node. The target pitch is reached by a one-pole glide whose rate grows with rotation speed and jumps at motion onset, so faster motion moves to the new root more quickly (Figure 4).

Kinetic timbre. *Azimut Kinetic* keeps the Azimut root and maps one smoothed rotation-energy value $E \in [0, 1]$ (gyroscope magnitude normalised over 30–750°/s) to many timbre parameters at once [7]: lowpass cutoff, partial amplitudes, drive, noise, vibrato, tremolo, and reverb. A still staff sounds close to a pure sine; fast spins sound buzzy, noisy, and reverberant. Each completed lap also

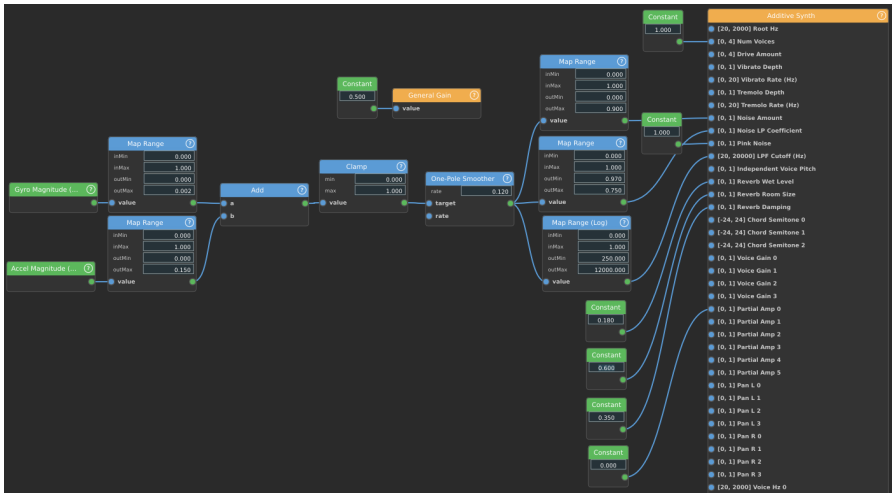


Figure 3: The complete *Wind Noise* preset in the plugin. Gyroscope and acceleration magnitudes are scaled, summed, clamped, and smoothed into one motion-energy signal that drives the noise level, noise filter, and lowpass cutoff of the additive engine (right; its unused inputs are cropped).

triggers a random sample-and-hold, whose values drift slowly into the same parameters, so two similar spins give two nearby but different timbres. The whole preset is built from existing math nodes.

4.6 Live Patching

The node-graph editor (Figure 4) is a pannable, zoomable canvas. Nodes are coloured by category, wires are drawn by dragging between typed pins, and new nodes come from a searchable, categorised picker. Hovering over any pin shows its current value, and Display nodes (number, meter, scope) can be placed on a wire to monitor it. A changed patch can be kept or reverted to its saved preset in one click.

Who patches, and when? A performer holding a 1.5 m staff in both hands cannot rewire anything in the middle of a kata, and a 100-node preset is not something to read at a glance. In practice the editor is an authoring tool. Between takes in rehearsal, the player puts the staff down, changes a range or swaps which feature drives a parameter, picks the staff up, and tries again, with audio running the whole time. In performance the player switches between prepared presets. A second person at the computer could edit while the player plays, but we have not tried that on stage.

4.7 Synthesis

Synthesis engines share one interface and are skipped entirely when the active preset does not wire them. Every preset except *Vocal Tract* drives the additive engine: four voices of six harmonic partials, with soft-clip waveshaping, vibrato, tremolo, chorus, filtered noise, a state-variable lowpass, and reverb. *Vocal Tract* drives a C++ port (via `cutelabnyc/pink-trombone-cpp`, MIT-licensed) of Neil Thapen's *Pink Trombone* [23]; only the control layer around it is ours. Engine parameters are smoothed per sample, so values that change once per packet do not cause zipper noise. If no packet arrives for 150 ms, the master gain fades out over 120 ms, and fades back in over 10 ms when packets return.

4.8 How the Software Got Here

We built the software in three steps. The first was a minimal OSC receiver with one hard-coded tilt-to-pitch mapping. The second added mappings as separate C++ classes on top of a shared analyzer, each one bringing new vocabulary (chords, energy features, facing, thrust). By the time there were eleven of them, it was clear that they differed only in how they wired together the same arithmetic, shaping, and stateful pieces. So the third step replaced the classes with the graph engine. Since then a second synthesis engine (§4.7) and four new presets have been added without touching the engine.

5 Playing the Bô: From Technique to Sound

We designed the staff-specific features around families of bô technique, and this is where the presets differ most from a generic motion controller.

Spins. Two-handed spins beside the body and overhead spins are the staff's most characteristic motions. Spin plane and direction select among four roots in *Martial Momentum*, so switching from a vertical clockwise spin to a horizontal one is heard as a change of harmony. In the Azimut presets, the length of a spin, through the lap count, is heard as a slow timbral sweep.

Changes of direction. A kata follows a floor pattern (*embusen*) with turns between facings. Facing is the third boolean of the Azimut root table, so the same spin played after a quarter turn lands on a different root, and a form traced across the floor becomes a sequence of harmonic regions. A half turn does not change the root, because facing only knows the line, not the side. Whether that is a limitation or a feature depends on the form, and we have not settled it.

Stances and transitions. Held guard positions (*kamae*) are almost static, so they show up as tilt and roll. In *Simple* and *Bowed*

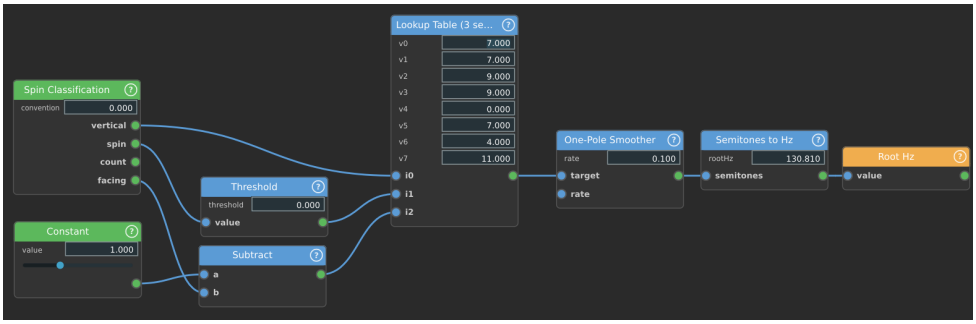


Figure 4: The node-graph editor showing the Azimut root-note chain: spin plane, direction, and facing select an entry of the 3-selector lookup table, which is smoothed and converted to the root frequency (green Source, blue Math, amber Sink).

Chord, the stance chooses the pitch or chord. In *Azimut Kinetic*, stillness gives a near-pure tone and energy brightens it, so the contrast between a held stance and the move out of it shapes the timbre directly.

Thrusts and strikes. Thrusts (*tsuki*) and strikes are short and decisive and mostly run along the staff. They are what the axial detector targets, and they are what pilot participants missed in the sound (§6.1).

6 Evaluation

The evidence so far comes from three sources: properties of the implemented system, simulation checks of the calibration, filter, and analyzer, and an exploratory pilot that was run before the fixes. Table 3 summarises it for each contribution.

6.1 Pilot Session

Ten adults with no prior exposure to REMORA took part in a short, informal pilot in August 2026 (see Ethical Statement). It used the earlier firmware and analyzer, before the changes in §6.2. Each participant was calibrated, given a brief demonstration, and then played freely across a subset of the nine selector presets before filling in an anonymous questionnaire. The group does not match the intended performer. Nine of the ten rated their martial-arts or staff experience 1 or 2 out of 5. General musical experience averaged 3.4/5, and prior experience with gestural instruments 2.6/5. The results therefore say something about how newcomers meet the instrument and little about how a trained staff player would use it.

We wrote the questionnaire for this pilot. It asked for agreement with six statements about the playing experience on a five-point scale (Table 4), followed by preset preference and free text. It is not a standard DMI evaluation instrument [16, 25], and with N=10 the ratings are only descriptive. Comfort scored highest, followed by novelty and absorption; predictability and expressiveness scored lower and varied more.

Preferences were spread out. *Azimut Kinetic* was chosen most often (3/10), then *Wind Noise* and *Lead+Drone* (2/10 each), and *Spin Voices (Scale)* and *Vocal Tract* once each. The reasons given followed each preset's character: *Azimut Kinetic* "matches best what I know from my martial art experience", *Wind Noise* needs "no knowledge [...] in martial art or music", and *Lead+Drone* made melody the easiest to steer. The tenth participant named no single preset ("All of them are great") and pointed to live patching instead ("the ability to customize live and create new patches directly inside with the node things is great"). Another asked for the wiring interface to be easier, "maybe some kind of tutorial".

Thrusts were the one complaint we had not anticipated. Three of ten participants said that thrusts which felt physically decisive did nothing to the sound ("The thrust are not well detected"; "Add 'thrust' detection support or fix the existing"). Other frustrations were described as learning ("when a bit hard it's just a learning experience"), and one participant found the instrument fun but not, on its own, motivating to practise.

6.2 What the Thrust Complaint Uncovered

Looking for the cause of the thrust problem, we found three defects. Each one affected more than thrusts.

Per-block analysis. The analyzer ran once per audio block on whichever packet was newest, with Δt from the host clock. At 48 kHz with 256-sample buffers, each 10 ms packet was analysed about twice. Every derivative-based feature saw one large step followed by a zero, its size depended on the buffer size, and the thrust detector could see a false zero-crossing of jerk. Packets are now queued from the network thread to the audio thread, and each one is analysed exactly once, with Δt from the sensor's clock. The mapping graph is evaluated once per packet too.

Gravity in the wrong frame. Dynamic acceleration was computed as the raw acceleration, in the chip's frame, minus gravity rotated into the calibrated staff frame. Those two frames differ by the board's alignment and by the mirrored axis described in §4.2. A still, tilted staff therefore showed up to 2 g of "motion" (§6.3). The threshold for a thrust is 1.5 g, so the detector's input was largely a tilt signal. We now think this, more than scheduling, is why thrusts went unheard. It also fed into Energy and Jerk. Both vectors are now rotated into the world frame before they are subtracted.

Tilt lost during spins. With the library's fixed-gain filter, centripetal acceleration pulled the estimated vertical towards the staff during fast spins, and spin plane, facing, and the tip vector all inherit that error. The gated filter (§4.1) and the gyroscope-based rotation axis (§4.3) address it.

The facing fix (§4.3) belongs with these too. Energy and Jerk now have a different scale at rest, so the presets that use them will need to be checked again by ear on the staff.

Table 3: Contributions, current evidence, and open questions.

Contribution	Current evidence	Untested
(1) bô features	spin- and facing-driven *Azimut Kinetic* was the preset most often preferred in the pilot; in simulation, facing holds through overhead spins and changes once per quarter turn, and features no longer depend on buffer size (§6.3)	accuracy on the real staff; yaw drift over a set; thrust detection on labelled real thrusts
(2) calibration	in simulation, tip error $< 0.03°$ and identical roll over 192 mounts (§6.3)	remounting at several angles on hardware
(3) mapping graph	used throughout development; all eleven hand-coded mappings and four new ones built from one catalog (§4.5); one pilot participant singled out live patching	whether other people can author with it unaided; mappings designed independently of the catalog

Table 4: Agreement on a 1–5 scale, N=10: mean (sample SD), median, and range.

Statement	Mean (SD)	Mdn	Range
Predictability ("in control, could predict")	3.4 (1.3)	4	1–5
Physical comfort	4.0 (1.1)	4	2–5
Novelty ("new ways to make sound")	3.9 (1.0)	4	2–5
Expressiveness through movement	3.3 (1.3)	3	1–5
Absorption ("less about how, more about music")	3.9 (1.0)	4	2–5
Musical satisfaction	3.6 (1.3)	4	1–5

6.3 Checks in Simulation

Each of the fixes above can be checked without the staff. A tool in the repository runs the plugin's own calibration and analyzer code, and the firmware's filter, on simulated motion: ideal kinematics plus sensor noise (0.01 g and 0.3°/s standard deviation), with the sensor 0.7 m from the hands. These checks show that the code does what we intended. They do not show that it works on a real staff, where hands, staff flex, and sensor errors are messier. The main results:

- **Calibration.** We tested all 24 axis-aligned board orientations, at 8 roll angles each (192 mounts), with a random filter heading and 50 random staff orientations per mount. The worst tip error was 0.03°. The same staff orientation read the same roll on every mount. With every pose up to 10° off, no calibration was rejected, but the tip error grew to 18°, so careful poses matter.
- **Filter during spins.** The worst tilt error during a 4 s spin was as follows. For a vertical-plane spin at 1 rev/s: 179° with fixed gain, 1.8° gated. At 2 rev/s: 179° against 3.6°. For an overhead spin at 2 rev/s: 81° against 0.0°. With fixed gain the vertical is simply lost. Both filters recover once the staff is still.
- **Gravity removal.** On a still staff, over 24 mounts and 20 orientations each, the previous formula left a residual of 1.29 g on average and 2.00 g at worst. The current one leaves none.
- **Scheduling.** We simulated twenty thrusts of 2–4 g at random headings and tilts, then 20 s of spins and swings as negatives. Per-packet analysis found 20/20 thrusts with no false detections, with identical output at buffer sizes of 64 to 1024 samples. Scheduled per block, with the gravity fix already in place, the same analyzer also found all 20. Its onset peak, however, varied from 0.27 to 0.55 with buffer size, against a constant 0.26 per packet.
- **Facing.** Over a 5 s overhead spin at 2 rev/s, facing changed 0 times; the previous rule changed it 40 times. A quarter turn of the body changed it exactly once, both when the staff was held still and when it went from a side spin to a spin in the other plane.

6.4 Latency

We have not measured end-to-end latency. It can be broken down into parts, two of which are unknown. The sample can be up to 5 ms old (200 Hz IMU), and it may wait up to 10 ms for the next send. Wi-Fi delivery comes next, and it is not yet measured. Once a packet arrives, it waits up to one block (5.3 ms at 256 samples and 48 kHz) to be analysed, and the result then goes through the host's output buffer, another 5.3 ms plus whatever the audio device adds. Without the network and the device, that is 5–26 ms. The figure of about 50 ms in an earlier draft was a guess, and we withdraw it. Most presets map continuous motion to gliding parameters, which tolerate delay better than percussive events do. Thrusts are the exception, because their sound should land with the impact, and even the known part of the budget can exceed the 10 ms often cited for intimate control [27]. The plugin's packet log and live jitter display make the network part easy to measure, and a contact microphone on the staff recorded alongside the plugin's output will give the total.

6.5 Not Yet Measured

These items are planned and need the real staff or other people. They are not part of the evidence above. (i) End-to-end latency and network jitter and loss, measured as described above. (ii) A labelled thrust test on the real staff: N thrusts and N non-thrust strikes, counted as detected or missed from a packet log replayed through the tool. (iii) Yaw drift over a 20–30 minute set, and the tilt error during spins against an optical reference. (iv) A remount test of the calibration on hardware. (v) A timed, unassisted rewiring task. (vi) Sessions with participants trained in bô or another martial art. (vii) Battery runtime.

7 Discussion

7.1 Limitations

Evidence. Every fix in §6.2 has so far been checked only in simulation, and the only user data predates the fixes and comes mostly from novices.

Heading. Facing depends on a heading with no absolute reference. Bias tracking slows drift and the re-zero button resets it, but the button needs a hand free, and we have no gesture for it yet.

Sensor position. Calibration corrects rotation but not position along the staff, so the acceleration-based features depend on where the unit is mounted.

Network. The hotspot name and password are compiled into the firmware, so pairing with a different host means reflashing over USB. A laptop hotspot limits range and is sensitive to interference. There is no host discovery beyond sending to the gateway, and the only feedback when joining fails is a blinking LED.

Interface. The graph editor serves patch authors better than performers who only want to choose a preset. There is no simplified performance view.

7.2 Future Work

The immediate work is the measurement list in §6.5, together with re-tuning the presets by ear on the corrected features. Replacing the hotspot with ESP-NOW, sending to a second ESP32 on the host's USB port, would take network credentials out of the firmware. Longer term, we plan a dual-IMU configuration, one sensor at each end of the staff, which would also let centripetal acceleration be separated from linear motion. We also plan gesture-triggered preset switching and a heading re-zero gesture, and a shareable patch format.

8 Conclusions

REMORA makes a bô into a musical instrument whose features (spin plane, spin direction, the line the staff works along, lap count) come from the way a staff is handled. Its mapping is a patch that can be edited while audio runs, and all of the project's hand-coded mappings now live as presets of one node catalog. First-time players in a ten-person pilot found it comfortable and engaging and most often preferred a spin- and facing-driven preset. Their one clear complaint, that thrusts were not heard, led us to three defects that a stock orientation filter and a per-block analyzer introduce on a long spinning object. We fixed them and checked the fixes in simulation. What remains is to show the same on the real staff, with trained players.

Acknowledgments

This work was carried out during the first author's internship at CREATE, Aalborg University, Denmark.

9 Supplementary Material

- **Source and design files:** firmware, plugin source, OpenSCAD enclosure, all fifteen preset graphs, the tool that counts each preset's nodes, and the simulation check and log-replay tool used in §6.3: https://github.com/TimotheeDvt/REMORA

10 Ethical Statement

Accessibility. REMORA assumes the mobility and two-handed grip strength needed to swing a 1–1.5 kg staff, and no reduced-effort physical variant exists. The skill barrier varies across presets (*Simple* needs only tilt and roll), and a facilitator can patch a simpler mapping for a newcomer, but this does not address the physical requirement.

Environmental impact. The enclosure is attached with zip ties, so the electronics and battery can be removed and reused. No lifecycle analysis has been performed.

Socio-economic fairness. REMORA uses free and open-source tools (JUCE, PlatformIO, OpenSCAD) and inexpensive, widely available hardware, and all source and design files are public.

Human participants. The pilot (§6.1) involved ten adult volunteers who answered an anonymous questionnaire. No names or identifying data were recorded, free-text answers are quoted without attribution, and participants could stop at any time. Each participant gave verbal consent before the session.

Use of Artificial Intelligence. We disclose that a locally-run large language model (Mistral 7B) was used to help draft and edit portions of this manuscript's prose under the first author's direction and review. This disclosure applies only to the writing of this paper: REMORA's firmware, plugin, and mapping code were designed and written entirely by the first author without AI code generation.

References

[1] Andrew Belt and VCV. 2017. VCV Rack: Open-Source Virtual Modular Synthesizer. https://vcvrack.com. Software, first released September 2017.

[2] Antonio Camurri, Shuji Hashimoto, Matteo Ricchetti, Andrea Ricci, Kenji Suzuki, Riccardo Trocca, and Gualtiero Volpe. 2000. EyesWeb: Toward Gesture and Affect Recognition in Interactive Dance and Music Systems. *Computer Music Journal* 24, 1 (2000), 57–69. doi:10.1162/014892600559182

[3] Jean-Michaël Celerier, Pascal Baltazar, Clément Bossut, Nicolas Vuaille, Jean-Michel Couturier, and Myriam Desainte-Catherine. 2015. OSSIA: Towards a Unified Interface for Scoring Time and Interaction. In *Proceedings of the International Conference on Technologies for Music Notation and Representation (TENOR)*. Paris, France.

[4] Sarah Fdili Alaoui, Jules Françoise, Thecla Schiphorst, Karen Studd, and Frédéric Bevilacqua. 2017. Seeing, Sensing and Recognizing Laban Movement Qualities. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 4009–4020. doi:10.1145/3025453.3025530

[5] Rebecca Fiebrink and Perry R. Cook. 2010. The Wekinator: A System for Real-time, Interactive Machine Learning in Music. In *Late-Breaking Demo, International Society for Music Information Retrieval Conference (ISMIR)*. Utrecht, Netherlands. https://ismir2010.ismir.net/proceedings/late-breaking-demo-13.pdf

[6] Jules Françoise, Norbert Schnell, Riccardo Borghesi, and Frédéric Bevilacqua. 2014. Probabilistic Models for Designing Motion and Sound Relationships. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. London, UK, 287–292. doi:10.5281/zenodo.1178764

[7] Andy Hunt and Marcelo M. Wanderley. 2002. Mapping Performer Parameters to Synthesis Engines. *Organised Sound* 7, 2 (2002), 97–108. doi:10.1017/S1355771802002030

[8] Sebastian Madgwick and Thomas Mitchell. 2013. x-OSC: A Versatile Wireless I/O Device for Creative/Music Applications. In *Proceedings of the Sound and Music Computing Conference (SMC)*. doi:10.5281/zenodo.850439

[9] Robert Mahony, Tarek Hamel, and Jean-Michel Pflimlin. 2008. Nonlinear Complementary Filters on the Special Orthogonal Group. *IEEE Trans. Automat. Control* 53, 5 (2008), 1203–1218. doi:10.1109/TAC.2008.923738

[10] Joseph Malloch, Stephen Sinclair, and Marcelo M. Wanderley. 2015. Distributed Tools for Interactive Design of Heterogeneous Signal Networks. *Multimedia Tools and Applications* 74, 15 (2015), 5683–5707.

[11] Joseph Malloch and Marcelo M. Wanderley. 2007. The T-Stick: From Musical Interface to Musical Instrument. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. New York, USA, 66–69. doi:10.1145/1279740.1279751

[12] Teresa Marrin and Joseph Paradiso. 1997. The Digital Baton: A Versatile Performance Instrument. In *Proceedings of the International Computer Music Conference (ICMC)*. Thessaloniki, Greece. https://quod.lib.umich.edu/i/icmc/bbp2372.1997.

083

[13] Teresa Marrin and Rosalind W. Picard. 1998. The "Conductor's Jacket": A Device for Recording Expressive Musical Gestures. In *Proceedings of the International Computer Music Conference (ICMC)*. Ann Arbor, MI, USA. https://www.media.mit.edu/publications/the-conductors-jacket-a-device-for-recording-expressive-musical-gestures-2/

[14] Max V. Mathews. 1991. The Radio Baton and Conductor Program, or: Pitch, the Most Important and Least Expressive Part of Music. *Computer Music Journal* 15, 4 (1991), 37–46.

[15] Sangbong Nam. 2013. Musical Poi (mPoi). In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Daejeon, Korea, 148–151. https://www.nime.org/proceedings/2013/nime2013_254.pdf

[16] Sile O'Modhrain. 2011. A Framework for the Evaluation of Digital Musical Instruments. *Computer Music Journal* 35, 1 (2011), 28–42.

[17] Zeynep Özcan and Anıl Çamcı. 2024. Juggling for Beginners: Embracing and Fabricating Failure as Musical Expression. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Utrecht, Netherlands. https://nime.org/proceedings/2024/nime2024_21.pdf

[18] Miller Puckette. 1988. The Patcher. In *Proceedings of the International Computer Music Conference (ICMC)*. San Francisco, CA, USA, 420–429.

[19] Miller Puckette. 1996. Pure Data: Another Integrated Computer Music Environment. In *Proceedings of the Second Intercollege Computer Music Concerts*. Tachikawa, Japan, 37–41.

[20] Miller Puckette. 2002. Max at Seventeen. *Computer Music Journal* 26, 4 (2002), 31–43. doi:10.1162/014892602320991356

[21] Jan C. Schacher. 2010. Motion to Gesture to Sound: Mapping for Interactive Dance. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Sydney, Australia, 250–254. https://www.nime.org/proceedings/2010/nime2010_250.pdf

[22] Masami Takahata, Kensuke Shiraki, Yutaka Sakane, and Yoichi Takebayashi. 2004. Sound Feedback for Powerful Karate Training. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Hamamatsu, Japan, 13–18. doi:10.5281/zenodo.1176673

[23] Neil Thapen. 2017. Pink Trombone: A Voice Synthesizer. https://dood.al/pinktrombone/. Web-based digital-waveguide vocal tract physical model.

[24] Doug Van Nort, Marcelo M. Wanderley, and Philippe Depalle. 2014. Mapping Control Structures for Sound Synthesis: Functional and Topological Perspectives. *Computer Music Journal* 38, 3 (2014), 6–22. doi:10.1162/COMJ_a_00253

[25] Marcelo Mortensen Wanderley and Nicola Orio. 2002. Evaluation of Input Devices for Musical Expression: Borrowing Tools from HCI. *Computer Music Journal* 26, 3 (2002), 62–76.

[26] Johnty Wang, Joseph Malloch, Stephen Sinclair, Jonathan Wilansky, Aaron Krajeski, and Marcelo M. Wanderley. 2019. Webmapper: A Tool for Visualizing and Manipulating Mappings in Digital Musical Instruments. In *Proceedings of the 14th International Symposium on Computer Music Multidisciplinary Research (CMMR)*. Marseille, France.

[27] David Wessel and Matthew Wright. 2002. Problems and Prospects for Intimate Musical Control of Computers. *Computer Music Journal* 26, 3 (2002), 11–22. doi:10.1162/014892602320582945